FDE ACADEMY · LESSON 7 OF 9

Observability

For any decision: why did it take this long, which tool failed, how many tokens did it use, and which model made the call?

Raunak Sharan
Enterprise Agentic AI Engineering

git checkout lesson-07

OBJECTIVES

By the end of today you can

01

Trace a decision: one root span, a child for every node, check, retrieval, model call and tool

02

Record model calls with the GenAI conventions, never the prompt or reply

03

Carry trace context into worker threads, and explain what breaks without it

04

Meter every decision: calls, tokens, cost, time per step

05

Keep personal data out of prompts, messages and spans, and prove it

06

Read a trace to find where the time went, including time no step owns

FOUR KINDS OF RECORD

Different readers, different rules

Record	Answers	Who reads it	Kept
Audit trail	Why was this decided?	Risk, compliance, regulators	Years, with the decision
Logs	What happened in this process?	Engineers on call	Days to weeks
Traces	Where did the time go in this request?	Engineers	Sampled, days
Metrics	How is the whole fleet doing?	Dashboards, alerts	Aggregated, months

Traces go to more systems and more people than the audit trail. So the audit can hold the evidence; traces never hold personal data.

UNDERWRITER TRACE APP-002

One tree per decision

```
underwrite APP-002 16.9 ms decision=MANUAL_REVIEW
├─ node income 0.5 ms outcome=ok attempts=1
├─ node checks 0.7 ms outcome=ok attempts=1
│  ├─ check credit_bureau 0.1 ms status=OK required=True
│  └─ … 3 more checks
├─ node policy 1.4 ms 3 × retrieve
├─ node agent 6.3 ms outcome=ok
│  ├─ chat mock-underwriter-v1 0.2 ms in=715 out=16
│  ├─ tool check_policy 4.6 ms outcome=ok
│  │  └─ retrieve 4.5 ms results=3
│  └─ chat mock-underwriter-v1 0.2 ms in=952 out=47
└─ node engine 0.7 ms decision=MANUAL_REVIEW
```

MANUAL_REVIEW · 2 LLM calls · 1667 in / 63 out · $0.000000 · 16.9 ms, 10.4 ms in nodes

THE MODEL CALL · LLM/INSTRUMENTED.PY

Record the shape of the call, never its words

On the span

```
gen_ai.operation.name = chat
gen_ai.system
gen_ai.request.model
gen_ai.request.max_tokens
gen_ai.usage.input_tokens
gen_ai.usage.output_tokens
llm.cost_usd · error.type
```

Never on the span

- The prompt or the reply
- Names, ids, emails, dates of birth, addresses
- Free-text error messages, unredacted
- Anything a support engineer shouldn’t read

CONTEXT PROPAGATION · WORKFLOW/RESILIENCE.PY

Worker threads don’t inherit the trace

Before: spans start in a fresh context

```
_pool.submit(attempt_fn)

node income ← no parent
check … ← no parent
```

After: run inside a copy of this context

```
_pool.submit(
contextvars.copy_context().run,
attempt_fn)
underwrite → node → check
```

The same copy carries the usage meter, so tokens from a thread land on the right decision.

METERING · OBSERVABILITY/METERING.PY

Every decision carries its own bill

2	1,667	$0	10.4
LLM calls	tokens in (63 out)	the mock is free	ms in nodes

- **decision.usage**: calls, tokens, cost, by model, time per step
- **PRICES** turns tokens into dollars; an unpriced model sets **cost_known = false**, never a silent $0
- The eval now reports calls and cost per case, and fails a decision with no usage record

TWO FINDINGS FROM BUILDING THIS LESSON

Looking at telemetry showed where data had already gone

APP-003 · ID digits

The bureau’s own error, “no record for applicant ending ####”, became a signal. Signals reach the model context, and now spans.

Fix: our message names no one.

APP-006 · two names

The identity-mismatch signal quoted the payslip name and the application name, so both went to the model.

Fix: a generic message; the names moved to evidence on the audit trail.

The lesson 3 test checked one sample, which had neither problem. It now checks all eight, for the model context and for every span.

READING A BAD DAY

Every fault from lessons 3–5 has a shape in the trace

Fault	What the trace shows
bank_data=server_error	× on check bank_data under node checks; the decision still approves
income=transient	node income · attempts=2 outcome=retried
--llm-fault loop	Five chat spans and four tool spans: the cost of a loop, visible at a glance
LLM unavailable	× chat span with error.type; the decision goes to a human
No fault at all	About 20 ms total, 11–15 ms in nodes: the rest is checkpointing between steps

An error in telemetry isn’t an error in the decision. Escalation is the system working.

LIVE WALKTHROUGH

Trace it, break it, read the tree

```
uv run underwriter trace data/applications/APP-002.json
uv run underwriter trace …/APP-001.json --tool-fault bank_data=server_error
uv run underwriter trace …/APP-001.json --node-fault income=transient
uv run underwriter trace …/APP-001.json --llm-fault loop
uv run underwriter run data/applications/APP-002.json # Usage: line
```

```
OTEL_EXPORTER=none | console | memory | otlp
# otlp: uv sync --extra otel, then Jaeger on :16686
```

LAB · 100 MINUTES

Three exercises

30 min

The slow span

The retrieve under tool check_policy is several times slower than the three under node policy. Find out why using spans only. Fix it, or say why it doesn’t matter.

40 min

Fleet metrics

Add a decisions counter by recommendation, a duration histogram and a token-usage counter by model. Test them with an in-memory reader.

30 min

A real price

Look up your production model’s price on the provider’s page, add it with the date, and work out the cost of one decision and of a looping agent.

HOMEWORK AND RUBRIC

One PR, three parts

- The slow-span explanation, and the fix if any
- The three metrics, with tests
- A price entry with its source and date, and a cost-per-decision note

Criterion	Strong	Weak
Coverage	Every node, tool, retrieval and model call is a span	Only the top level
Context	Correct parents across threads, tested	Orphan spans
Privacy	No PII in spans or prompts, tested on every sample	“We don’t log names”
Cost	Per decision, with unknown prices flagged	A monthly bill surprise

TAKEAWAY

Telemetry is another place data goes. Instrument everything, then look at what you sent.

Next · Lesson 8

Latency and resilience

Run the checks in parallel under one deadline, and retry with backoff and full jitter.